Telhado Solar Comunitário

Calculadora interativa de energia solar para comunidades de baixa renda

Disciplina: Coding for Security | FIAP 2026 | Global Solution — CitizenTech

1. Problema

Moradores de baixa renda no Brasil pagam proporcionalmente mais pela energia elétrica do que a classe média e alta. A tarifa social atinge apenas parte da população elegível, e muitos não conhecem alternativas como a energia solar fotovoltaica, que se tornou economicamente viável nos últimos anos.

A principal barreira não é tecnológica, mas informacional: o cidadão não sabe quanto custa um sistema solar, quanto economizaria por mês, em quanto tempo o investimento se paga, nem que é possível formar cooperativas solares para reduzir o custo por pessoa. Segundo a ANEEL, o Brasil atingiu 2 milhões de sistemas de geração distribuída em 2024, mas a concentração está em classes A e B.

2. Cidadão Beneficiado

O cidadão-alvo é o morador de comunidade, bairro periférico ou zona rural que paga conta de luz alta (R$150 a R$500/mês) e não tem acesso a informações sobre energia solar. Também atende líderes comunitários que podem organizar cooperativas solares em seus bairros.

Perfil: renda familiar de 1 a 3 salários mínimos, ensino médio completo ou incompleto, acesso a internet por celular. A interface foi projetada para ser simples, visual e autoexplicativa, com linguagem acessível e sem jargão técnico.

3. ODS Relacionados

ODS 13 — Ação contra a mudança global do clima

A energia solar reduz a emissão de CO2 ao substituir fontes fósseis. O sistema incentiva a adoção de energia limpa em comunidades que normalmente não teriam acesso a essa informação.

ODS 11 — Cidades e comunidades sustentáveis

Cooperativas solares tornam bairros mais sustentáveis e autossuficientes energeticamente, reduzindo a dependência da rede e a vulnerabilidade a aumentos tarifários.

ODS 10 — Redução das desigualdades

Democratiza o acesso à energia solar — hoje restrita às classes A e B — por meio de informação clara e do modelo cooperativo que divide custos.

4. Arquitetura e Modelo de Dados

Arquitetura

Aplicação web monolítica com separação em módulos Python:

- app.py: servidor Flask com 6 rotas (4 CRUD + cálculo + tarifas)
- database.py: conexão MySQL com CRUD via queries parametrizadas (sem ORM)
- external_api.py: integração com Open-Meteo (geocoding + irradiação solar)
- solar_calc.py: lógica de dimensionamento e economia
- llm_module.py: geração de relatório com LLM (Groq) ou template fallback

- logger_module.py: logging em arquivo com data/hora/ação/resultado
- Frontend: HTML5 + CSS3 + JavaScript puro com SVG interativo

Modelo de Dados — Tabela simulacoes

Campos: id (PK auto_increment), cidade (VARCHAR 100), estado (CHAR 2), latitude/longitude (DECIMAL 10,6), valor_conta (DECIMAL 10,2), tarifa_kwh (DECIMAL 10,4), consumo_kwh (DECIMAL 10,2), irradiacao_media (DECIMAL 10,4), num_placas (INT), custo_estimado (DECIMAL 12,2), economia_mensal (DECIMAL 10,2), tempo_retorno_meses (INT), num_moradores (INT default 1), custo_por_pessoa (DECIMAL 12,2), relatorio_llm (TEXT), created_at/updated_at (TIMESTAMP).

API Externa — Open-Meteo (gratuita, sem chave)

- Geocoding: converte nome da cidade em latitude/longitude
- Forecast: irradiação solar (shortwave_radiation_sum) dos últimos 30 dias, convertida de MJ/m² para kWh/m²/dia (fator 0.2778)
- Current Weather: temperatura, umidade e cobertura de nuvens (dados complementares)

Fluxo de Cálculo

- Usuário informa cidade, estado e valor da conta de luz
- Sistema busca coordenadas e irradiação solar via Open-Meteo
- consumo_kWh = valor_conta / tarifa_estado (dados ANEEL)
- energia_placa_mês = potência(0.55kW) × horas_sol × 30 × eficiência(0.80)
- num_placas = consumo / energia_placa_mês (arredondado para cima)
- Custo total = num_placas × R$1.800 | Economia = 90% da conta | Retorno = custo/economia

5. Segurança e Privacidade

Medidas Implementadas

- Credenciais (senha MySQL, chave LLM) em .env, fora do versionamento (.gitignore)
- Todas as queries SQL usam parâmetros (%s), prevenindo SQL Injection
- Validação de toda entrada: tipo, tamanho, intervalo (valor_conta > 0 e < 50000)
- Mensagens de erro genéricas — erros internos logados mas não expostos
- Coleta mínima: apenas cidade, estado e valor da conta (sem CPF, nome ou endereço) — LGPD

Três Ameaças e Mitigações

Ameaça 1: SQL Injection

Risco: atacante insere comandos SQL nos campos (ex: cidade = "'; DROP TABLE simulacoes;"). Mitigação: queries parametrizadas com %s do mysql-connector-python. Valores são dados, nunca parte do comando SQL. Campo estado validado para 2 caracteres.

Ameaça 2: Exposição de credenciais no código-fonte

Risco: credenciais hardcoded dariam acesso ao banco a qualquer pessoa com o repositório. Mitigação: credenciais no .env (no .gitignore). Repositório contém apenas .env.example com placeholders.

Ameaça 3: Abuso da API (rate limiting / DDoS)

Risco: milhares de requisições ao /api/calcular sobrecarregam servidor e Open-Meteo. Mitigação: validação rejeita requests malformados. Em produção: flask-limiter (10 req/min/IP) e CORS restrito.

6. Validação e Limitações

A solução foi validada com dados reais da API Open-Meteo para São Paulo/SP. A irradiação retornada (4.8 kWh/m²/dia) é consistente com o Atlas Solarimétrico do INPE para o Sudeste (4.5 a 5.5 kWh/m²/dia). Teste: conta de R$300/mês em SP resulta em 6 placas de 550W, economia de R$270/mês e retorno em 40 meses (3.3 anos).

Limitações

- Tarifas são médias estaduais (ANEEL), não a tarifa exata da distribuidora
- Cálculo não considera sombreamento, orientação do telhado ou manutenção
- Custo por placa (R$1.800) é média e varia por região
- Em produção: necessária autenticação e rate limiting
- Módulo LLM depende de API externa (Groq); sem chave, usa template local

7. Uso de IA Generativa

Este projeto utilizou IA generativa (Claude, Anthropic) como ferramenta de apoio para: estruturação do código e boas práticas de segurança, depuração de erros, e geração de ideias para a interface visual interativa (SVG da casinha com placas solares). Todo o código foi revisado, compreendido e pode ser explicado na arguição.

O módulo LLM integrado (llm_module.py) utiliza a API do Groq (modelo Llama 3.3 70B) para gerar relatórios explicativos em linguagem simples. Na ausência da chave de API, o sistema gera um relatório template funcional.